

2025 · TEAM PROJECT

Cooking Star.

요리 커뮤니티 웹 서비스 — 기록 · 공유 · 탐색 · 구매를 잇는 통합 플랫폼

Java 21

Spring Boot

Spring Security

MyBatis

PostgreSQL

JSP · JSTL

Gradle

PRESENTED BY

송태현
조종현

Backend / Full-stack Team Project · 2026

6+ Domains · 5 External APIs · Real Service Flow

프로젝트 개요

조회 중심의 단편적 레시피 서비스를 넘어, **기록 · 상호작용 · 탐색 · 구매**를 하나의 흐름으로 연결한 요리 커뮤니티 웹 서비스.

① CONTEXT

문제 인식

기존 레시피 서비스는 조회 중심이라 사용자 기록, 커뮤니티 반응, 검색·구매 연결성이 약하다고 판단.

② GOAL

서비스 목표

레시피 작성과 요리자랑, 댓글·좋아요·북마크·팔로우를 결합해 사용자가 **다시 방문할 이유**를 만들었습니다.

③ EXPANSION

확장 방향

검색 API, 맛집 저장, 상품 검색, 장바구니, 결제 내역까지 연결해 **생활형 요리 플랫폼**으로 확장.

12+

핵심 기능 · 회원 · 레시피 · 상호작용 · 구매

ROLE

권한 설계 · MEMBER · MANAGER · ADMIN

AJAX

비동기 UX · 좋아요 · 팔로우 · 장바구니

API · 5

Naver · Kakao · Gemini · Toss

기획 배경 & 문제 정의

단순 레시피 조회를 넘어, 사용자 참여형 커뮤니티가 필요했습니다.

AS-IS · 기존 서비스의 한계

기능이 분리되어 흐름이 끊긴다

- 01 레시피 **조회** 중심으로 끝나 사용자 기록과 반응이 남지 않음
- 02 탐색(레시피) — 구매(쇼핑) — 외출(맛집) 경험이 **서로 다른 앱**에 분산
- 03 **좋아요·북마크·팔로우** 같은 커뮤니티 반응이 없어 재방문 동기가 약함
- 04 "오늘 뭐 먹지" 같은 일상 문제에 **맞춤 추천**이 부재



TO-BE · Cooking Star의 방향성

하나의 서비스로 이어지는 경험

- ① **기록** — 레시피 등록 + 요리자랑 게시판으로 사용자 흔적 축적
- ② **공유** — 댓글·좋아요·북마크·팔로우로 사용자 간 연결
- ③ **탐색** — Naver / Kakao API로 외부 정보까지 한 화면에서 탐색
- ④ **구매·추천** — Toss 결제 + Gemini 메뉴 추천으로 일상까지 연결

사용자별 서비스 흐름

권한(ROLE)에 따라 접근 가능한 기능을 분리해 서비스 사용 흐름을 설계.

LANE 01

비회원

Guest

메인 페이지 → 레시피 조회 → 레시피 검색 (Naver) → 맛집 검색 (Kakao) → 실시간 검색어 랭킹 → 회원가입 / 로그인

LANE 02

회원

ROLE_MEMBER

레시피 작성·수정·삭제

요리자랑 게시판

댓글

좋아요

북마크

팔로우 / 언팔로우

마이페이지

최근 본 레시피

맛집 저장

상품 검색

장바구니

Toss 결제

구매내역

LANE 03

관리자

MANAGER · ADMIN

관리자 대시보드

회원 관리

레시피 관리

권한 변경

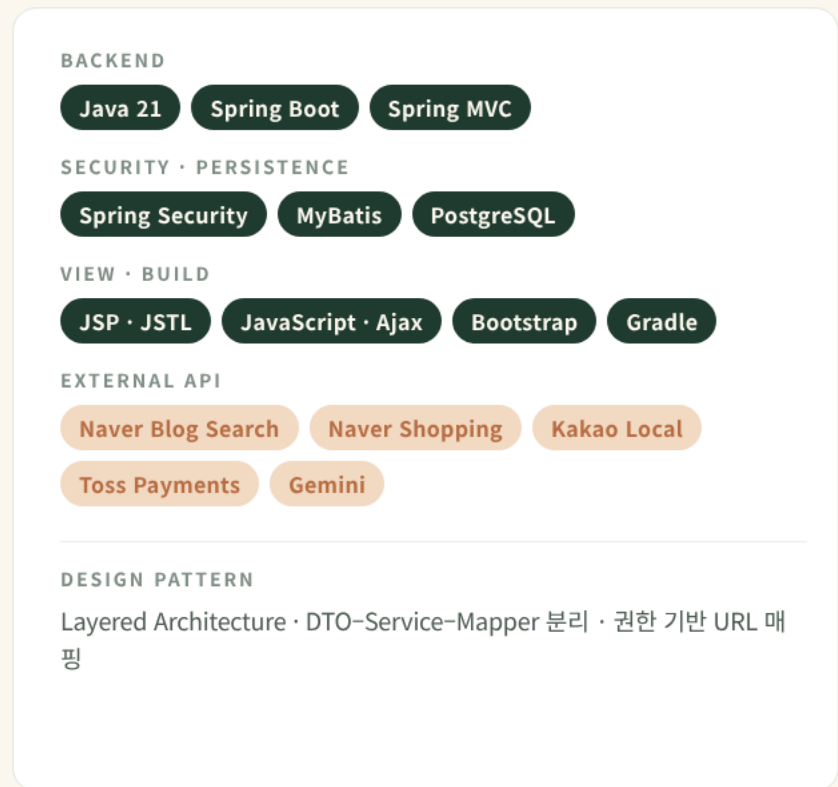
전체 통계 조회

기술 스택 & 아키텍처

SPRING MVC LAYERED ARCHITECTURE



TECH STACK



담당 역할 & 구현 범위

인증·인가부터 커뮤니티 상호작용, 외부 API 결과 저장까지 — 백엔드 전반에 걸쳐 기능을 설계·구현.



DOMAIN 01

인증 · 인가 · 마이페이지

회원가입 / 로그인 / 로그아웃 / Remember-me · Spring Security 인증 설정 · ROLE_MEMBER 자동 부여 · 마이페이지 · 프로필 이미지 등록·수정



DOMAIN 02

관리자 · 권한별 접근 제어

관리자 페이지 구성 · requestMatchers로 URL별 권한 분리 · MANAGER / ADMIN 권한 분기 · 레시피 삭제 권한 처리



DOMAIN 03 · 핵심

커뮤니티 상호작용

댓글 · **좋아요 토글** · 북마크 · 팔로우 / 언팔로우 · Ajax 기반 비동기 처리 · 인증 정보로 username 주입



DOMAIN 04

요리자랑 게시판 CRUD

작성 · 목록 · 상세 · 수정 · 삭제 · 조회수 · 좋아요 · 날짜·좋아요 수 정렬 · 메인 페이지 인기글 노출



DOMAIN 05

맛집 · 검색 API 저장

Kakao Local 맛집 검색 + 저장 · Naver 검색 API 결과 저장·삭제 · Set 기반 저장 여부 비교로 중복 방지



DOMAIN 06

안정화 · 문서화

방문자 수 · 전체 회원 수 통계 · URL 권한 구조 정리 · 기능 문서화 · 프로젝트 오류 해결 및 기능 안정화

회원 · 권한 처리

Spring Security 기반 인증·인가·URL별 접근 제어 · 회원가입 시 기본 권한 자동 부여.

ROLE-BASED ACCESS CONTROL

ROLE	대표 URL · 접근 가능 기능	부여 시점
PUBLIC	/ · /member/join · /member/login · 레시피·맛집 검색 전체 허용 — 로그인 없이 접근 가능	-
ROLE_MEMBER	/member/mypage · /recipe/insert · /community/* · /cart/* · 좋아요 · 북마크 · 팔로우 로그인 사용자가 사용하는 거의 모든 기능	회원가입 시 자동 부여
ROLE_MANAGER	/admin/recipe/* · 레시피 관리 · 부적절 콘텐츠 삭제 콘텐츠 운영 권한 (회원 권한 변경 불가)	관리자가 수동 부여
ROLE_ADMIN	/admin/** · 전체 회원 관리 · 권한 변경 · 통계 대시보드 최고 권한 — 모든 관리 기능 접근	초기 설정 시 부여

SECURITY CONFIG · CORE

```
@Bean
public SecurityFilterChain chain(HttpSecurity http) {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/", "/member/join",
                "/member/login")
            .permitAll()
            .requestMatchers("/admin/**")
            .hasRole("ADMIN")
            .requestMatchers("/admin/recipe/**")
            .hasAnyRole("ADMIN", "MANAGER")
            .anyRequest().authenticated()
        )
        .formLogin(...)
        .rememberMe(rm -> rm
            .tokenValiditySeconds(60 * 60 * 24 * 14))
        .logout(...);
    return http.build();
}
```

```
// 회원가입 시 자동 권한 부여
Member m = new Member(...);
m.setRole("ROLE_MEMBER");
memberMapper.insert(m);
```

레시피 CRUD & 파일 처리

작성·수정·삭제 시 **DB 데이터와 실제 파일 저장소의 동기화**를 핵심 과제로 정의.

CREATE
/recipe/insert



READ
/recipe/list · /detail



UPDATE
/recipe/update



DELETE
/recipe/delete

FILE HANDLING · 3 STRATEGIES

다중 이미지 업로드 정책

- 01 **CREATE** — 이미지 N장 업로드 → 서버 디스크 저장 + DB에 파일명 INSERT
- 02 **UPDATE** — 기존 - 삭제 + 신규 ≤ MAX 검증 후 처리 (이미지 개수 초과 방지)
- 03 **DELETE** — 파일 정보 조회 → DB 삭제 성공 → 실제 파일 삭제 (역순 보장)

DB ↔ FILE STORE SYNC

삭제 흐름 — 순서가 중요

STEP 1 SELECT — 삭제 대상 파일 정보 조회

STEP 2 DELETE FROM recipe / recipe_file — DB 삭제

STEP 3 DB 성공 시에만 File.delete() 호출

① DB 실패 시 파일은 유지되므로 데이터-파일 불일치를 최소화

요리자랑 게시판

사용자가 자신이 만든 요리를 자랑하는 커뮤니티 — 조회수·좋아요 기반 정렬로 메인 인기글 노출.

CORE FEATURES

담당 구현 범위

- ✓ **작성 / 수정 / 삭제**
권한 확인 후 CRUD 처리
- ✓ **좋아요 토글**
Ajax 비동기 · 상태 즉시 반영
- ✓ **날짜 / 좋아요 수 정렬**
ORDER BY 동적 SQL
- ✓ **이미지 첨부**
서버 디스크 + DB 파일명
- ✓ **상세 페이지 + 조회수**
상세 진입 시 view_count UPDATE
- ✓ **댓글**
로그인 사용자 한정 작성
- ✓ **메인 인기글 노출**
상위 N개 SELECT 후 메인 렌더
- ✓ **관리자 삭제**
ROLE_MANAGER / ADMIN 분기

DEFENSIVE DESIGN · CASE

최근 본 레시피 — null 방어

데이터 없을 때 DB까지 가지 않고 즉시 빈 배열 반환

```
@PostMapping("/recentList")
@ResponseBody
public List<RecipeDTO> recentList(
    @RequestParam("ids") List<Integer> ids) {

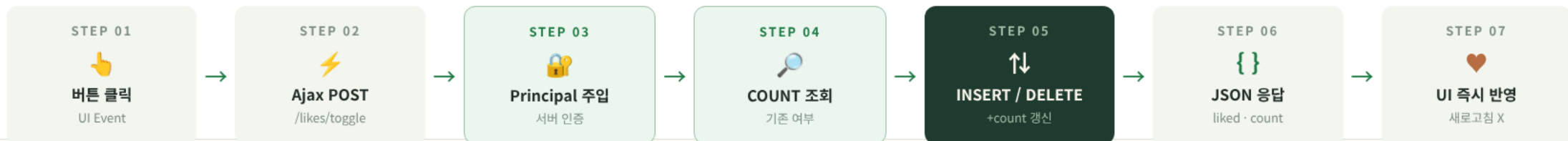
    // 유효성 검사: 데이터 부재 시 즉시 반환
    if (ids == null || ids.isEmpty()) {
        return new ArrayList<>();
    }

    return recipeService.findByIds(ids);
}
```

좋아요 토글 — 핵심 로직

새로고침 없이, 인증 정보로 username을 처리하고, 중복 좋아요 방지 + UI/DB 상태 동기화.

REQUEST FLOW · 7 STEPS



SERVICE · LIKE TOGGLE

@Transactional

```

public LikeResult toggle(long postId, String username) {

    // 1) 서버 인증 정보로만 username 처리 — 클라이언트 신뢰 X
    // 2) 기존 좋아요 여부를 COUNT(*)로 조회 (중복 방지)
    int exists = likeMapper.countByUserAndPost(username, postId);

    if (exists > 0) {
        likeMapper.delete(username, postId);           // 좋아요 취소
        postMapper.decreaseLikeCount(postId);           // count -1
        return new LikeResult(false, postMapper.getLikeCount(postId));
    } else {
        likeMapper.insert(username, postId);             // 좋아요 추가
        postMapper.increaseLikeCount(postId);           // count +1
        return new LikeResult(true, postMapper.getLikeCount(postId));
    }
}
  
```

COOKING STAR

// @Transactional → INSERT/DELETE + count UPDATE를 하나의 트랜잭션으로

KEY POINTS

새로고침 없는 UX

Ajax + JSON으로 버튼·카운트 즉시 갱신

서버 인증 기반 username

클라이언트가 보낸 값을 신뢰하지 않고 Principal에서 추출

중복 좋아요 방지

COUNT(*) 선조회 후 INSERT/DELETE 분기

트랜잭션 일관성

@Transactional로 INSERT/DELETE + count 갱신을 원자적으로 처리

커뮤니티 상호작용 기능

댓글 · 북마크 · 팔로우로 사용자 간 연결 구조를 만들고, 마이페이지에서 한눈에 확인.



댓글

COMMENT

로그인 사용자가 레시피·요리자랑에 댓글 작성 / 수정 / 삭제. 작성자만 수정·삭제 가능하도록 권한 검증.

@PreAuthorize

작성자 검증



북마크

BOOKMARK

관심 레시피를 저장. 좋아요와 동일한 토글 패턴 (COUNT 조회 → INSERT / DELETE) 으로 일관된 구조 유지.

Ajax 토글

마이페이지 노출



팔로우 / 언팔로우

FOLLOW

팔로워 ↔ 팔로잉의 양방향 관계 테이블로 사용자 간 그래프 구성. 자기 자신 팔로우 차단 + 중복 방지.

follow_user 테이블

자기참조 방지



마이페이지 집계

MY PAGE

팔로워 COUNT

팔로잉 COUNT

북마크 COUNT

내 레시피 COUNT

하나의 SELECT 또는 JOIN으로 통합 조회

외부 API 연동

RestTemplate 기반 호출 → JSON 파싱 → 서비스 로직과 연결. 외부 API 에러를 내부 에러와 **분리**해 추적성 확보.

API	사용 목적	적용 위치	통신 방식
Naver Blog Search	레시피 키워드 블로그 검색	메인·검색 페이지 · 저장 가능	REST · JSON
Naver Shopping	재료·상품 검색	상품 검색 → 장바구니 담기	REST · JSON
Kakao Local	주변 맛집 검색	맛집 검색 · 저장 (Set 기반 중복 방지)	REST · JSON
Toss Payments	결제 승인 · 검증	장바구니 → 결제 → 구매내역 저장	SDK + REST 콜백



HIGHLIGHT · AI

Gemini API · 시간대별 메뉴 추천

프롬프트 엔지니어링으로 출력 포맷을 단일 문장으로 강제 — 화면 일관성 유지

COMMON INTEGRATION PATTERN

RestTemplate → JSON → Map 추출 → DTO 매핑 → **HttpClientErrorException 분리**

외부 서버 상태 코드 에러를 내부 시스템 에러와 분리해 장애 원인 추적 용이

장바구니 & 결제 흐름

상품 검색부터 결제 완료, 구매내역 저장까지 — 실서비스형 커머스 흐름을 구현.

END-TO-END FLOW



핵심: 클라이언트의 amount를 신뢰하지 않고 서버 측 검증(Confirm)으로 결제 위변조 방지 · 결제 성공 시에만 cart 비움

관리자 기능

대시보드에서 전체 지표 확인 · 권한별 운영 범위를 분리해 안전하게 관리.

ADMIN DASHBOARD

/admin

통계 · 한눈에 보기

전체 회원
N
SELECT COUNT(*)
FROM member

오늘 방문자
N
visit_log
오늘 카운트

전체 레시피
N
SELECT COUNT(*)
FROM recipe

결제 건수
N
order 테이블
합계

MEMBER MANAGEMENT · /admin/member

USERNAME	NAME	ROLE	ACTION
user_a	홍길동	ROLE_MEMBER	권한 변경 · 삭제
user_b	이몽룡	ROLE_MANAGER	권한 변경 · 삭제
admin	관리자	ROLE_ADMIN	—

ROLE BOUNDARY

MANAGER vs ADMIN

MANAGER

콘텐츠 운영

- ✓ 부적절 레시피 / 게시물 삭제
- ✓ 신고 콘텐츠 검토
- × 회원 권한 변경 불가

ADMIN

최고 권한

- ✓ MANAGER의 모든 권한
- ✓ 회원 권한 변경 / 삭제
- ✓ 전체 통계 · 대시보드
- ✓ 시스템 설정

문제 해결 경험

개발 중 마주친 5건의 이슈 — 문제 → 원인 → 해결 → 결과 순서로 정리.

CASE A · UPDATE

레시피 수정 안정화

레시피 수정 시 파라미터 인식 문제

문제	myrecipe/update에서 DTO·파일·삭제 목록 처리 흐름이 분산되어 바인딩 불안정
원인	Controller가 파일 입출력까지 직접 다뤄 책임 분리 부재
해결	파일 처리 책임을 Service 계층으로 이관, 수정 대상 번호·파일 목록 명확 전달
결과	DB 데이터와 이미지 파일 처리 흐름 안정화

CASE B · DELETE

DB ↔ 파일 동기화

게시글 삭제 시 DB·파일 불일치 가능성

문제	DB row만 삭제되고 업로드 파일이 서버에 잔존할 수 있음
원인	파일 삭제 시점이 DB 삭제 이전·이후로 일관되지 않음
해결	SELECT 파일정보 → DB DELETE → DB 성공 시에만 실제 파일 삭제 순서 고정
결과	데이터·파일 저장소 불일치 가능성 축소

CASE C · IMAGE COUNT

검증 로직 추가

이미지 수정 시 업로드 개수 초과

문제	기존·삭제·신규 이미지를 함께 고려하지 않으면 MAX 제한이 깨질 수 있음
해결	기존 - 삭제 + 신규 ≤ MAX 검증 로직 추가
결과	수정 상황에서도 이미지 개수 제한 안정 유지

CASE D · SECURITY

URL 권한 분리

권한별 URL 접근 제어 정리

문제	로그인 사용자·일반 회원·매니저·관리자별 접근 URL 구분이 필요
해결	Spring Security <code>requestMatchers</code> 로 URL 분리, 회원가입 시 ROLE_MEMBER 자동 부여
결과	관리자·회원·공개 기능 접근 범위 명확화

프로젝트 성과

단순 CRUD를 넘어 실제 서비스 흐름을 구현 — 인증·파일·외부 API·결제·관리자까지 통합.

DOMAINS

6+

회원 · 레시피 · 커뮤니티 · 검색 · 구매 · 관리

EXTERNAL APIs

5

Naver · Kakao · Toss · Gemini

USER ROLES

4

PUBLIC · MEMBER · MANAGER · ADMIN

CORE FLOW

E2E

탐색 → 기록 → 상호작용 → 결제

ACHIEVEMENT 01

실서비스형 흐름 완성

- ✓ 비회원 탐색 → 회원 참여 → 결제까지 단절 없는 흐름
- ✓ 외부 검색 결과를 자체 서비스 내 저장·재활용
- ✓ Ajax + JSON으로 새로고침 없는 상호작용 UX

ACHIEVEMENT 02

계층 구조 + 책임 분리

- ✓ Controller → Service → Mapper 계층 분리 체득
- ✓ 파일 입출력·트랜잭션·외부 API 호출을 Service로 이관
- ✓ @Transactional로 INSERT/DELETE + count 갱신 원자화

ACHIEVEMENT 03

협업 · 문서화 경험

- ✓ 팀 분담 — 회원/관리/커뮤니티(송태현·조종현)
- ✓ URL · 권한 · 기능 문서화 (URL 정리본 산출)
- ✓ 디버깅 케이스 5건을 문제 → 해결로 기록

포트폴리오 관점의 핵심 역량

이 프로젝트를 통해 검증된 백엔드·풀스택 역량을 8개 영역으로 정리.

01

백엔드 CRUD 설계

레시피 · 요리자랑 · 댓글 · 좋아요 · 북마크 · 팔로우 ·
장바구니 · 구매내역 등 다도메인 CRUD

02

Spring Security 인증·인가

requestMatchers · ROLE 기반 접근 제어 ·
Remember-me · 회원가입 자동 권한 부여

03

MyBatis DB 연동

동적 SQL · IN() 절 방어로직 · JOIN/집계 쿼리 · 트랜잭
션 처리

04

Ajax 비동기 처리

좋아요·팔로우·장바구니 수량 등 새로고침 없는 UX ·
JSON 응답 → UI 즉시 반영

05

외부 API · JSON 처리

RestTemplate · 다중 계층 JSON → Map 추출 ·
HttpClientErrorException 분리

06

파일 업로드 · 저장소 관리

다중 이미지 업로드 · 수정 시 개수 검증 · DB ↔ 파일
동기화 순서 설계

07

권한별 서비스 분리

PUBLIC / MEMBER / MANAGER / ADMIN URL·기능
분리 · 권한 변경 시나리오 설계

08

오류 분석 · 해결

트러블슈팅 5건 기록 · 문제 → 원인 → 해결 → 결과 구조
로 설명 가능

Java · Spring · Spring Security · MyBatis · PostgreSQL · Ajax · REST API — 백엔드 중심 풀스택 전체 흐름 경험

Cooking Star는 단순 게시판이 아닌 사용자 흐름 중심의 웹 서비스.

NEXT · 향후 개선 방향

QUALITY

테스트 코드 추가

JUnit + Mockito · Service 단위 테스트부터 점진 도입

ARCHITECTURE

REST API 구조 전환

JSP 의존 축소 · SPA / 모바일 연동 대비

RELIABILITY

API 예외 처리 강화

@ControllerAdvice · 외부 API 재시도 · 서킷

PERFORMANCE

검색 결과 캐싱

동일 키워드 외부 API 호출 횟수 절감

OPS

배포 · 모니터링

Docker · Nginx · 헬스체크 · 메트릭 수집

OBSERVABILITY

서비스 로그 관리

구조화 로그 + 외부 API 호출 트레이싱

THANK YOU

송태현 / 조종현 · Cooking Star Team

Java 21

Spring Boot

MyBatis

PostgreSQL